🍱 Pattinathil-Pasi

Smart ML-Powered Excess Food Routing System

🎯 Core Idea (In Simple Words)

When a wedding hall, hotel, or event has **extra food**, instead of manually calling NGOs…

Our system:

1. Finds nearby NGOs
2. Uses Machine Learning to predict:
    - Who is most likely to accept
    - Who can arrive fastest
3. Ranks them intelligently
4. Sends alert only to the best ones
5. Assigns pickup automatically
6. Learns from every outcome

It is **not predicting waste.**
It is optimizing rescue in real time.

🧠 What Problem Are We Solving?

Current problem:

- Food providers don't know which NGO is available
- Calls are random
- Many NGOs ignore requests
- Food expires while coordination happens
- No performance tracking

Our solution:

> A real-time intelligent rescue routing engine.

👥 System Actors

🍽️ Food Providers

- Wedding halls
- Caterers
- Restaurants
- Event venues

🚚 NGOs

- Food redistribution NGOs
- Volunteer pickup teams
- Community kitchens

🛠️ Admin

- Retrains ML model
- Monitors performance
- Maintains NGO registry

🌐 Platform Type

Single web app with:

- Provider Dashboard
- NGO Dashboard
- Admin Panel

Built using:

- Frontend: React / Streamlit
- Backend: FastAPI (Python)
- ML: scikit-learn
- DB: PostgreSQL / Firebase

🔥 FULL END-TO-END WORKFLOW

🟢 STEP 1—Provider Creates Rescue Request (Trigger Event)

Provider logs in and submits:

- Meals available
- Food type
- Ready time - pickup time
- Expiry time - shelf life
- GPS location (auto captured)
- Event type
- Cause tag (why surplus happened)

📌 Cause Tag (Fairness Constraint)

Options:

- Overestimated attendance
- Guest no-show
- Weather issue
- Buffer cooking policy
- Unknown

This ensures:

- Providers are not unfairly penalized
- Surplus amount does NOT reduce score

Request becomes:

LIVE FOOD RESCUE EVENT

This event triggers the ML engine.

🟢 STEP 2 — Nearby NGOs Are Filtered

Backend does:

1. Reads provider GPS
2. Fetches NGO list
3. Filters NGOs within radius (e.g., 15 km)
4. Calculates distance using:
    - Haversine formula OR
    - Maps API

Now we have candidate NGOs.

🟢 STEP 3 — ML Feature Builder (Critical Part)

->Send to set of ngo's filtered by distance + reliability score

For each NGO candidate, system creates a feature vector.

🔍 ML Feature Constraints

Only use features available **before decision time**.

We use historical + contextual features:

distance_km
time_of_day
day_of_week
ngo_accept_rate
ngo_avg_response_time
Past_pickups (NGO)
Recent_activity_count - isactive?(0/1)
ngo_reliability_score = accept rate*avg resp time*past pickups

These are safe, realistic, and available in real time.

This prevents data leakage — very important in viva.

🟢 STEP 4 — ML Models Run

Model — Acceptance Probability Model

Type:
RandomForestClassifier

Input:
Feature vector above

Output
Probability NGO accepts quickly

Example output:
NGO A → 0.82
NGO B → 0.45
NGO C → 0.67

🟢 STEP 5 — Smart Ranking Engine

Now system calculates final score:

final_score =
0.5 × acceptance_probability
+ 0.3 × (1 / actual time collected)
+ 0.2 × reliability_index

This balances:

- Likelihood of accepting
- Speed
- Past reliability

Top 3:

-because to prevent far NGO's from taking the catering as if a far ngo takes it in then time may become high

Therefore for first 5 mins - immediately create jobs for notifying top 5 ngos

Next then 5 and finally all😸

NGOs selected.

This is your intelligent routing engine.

🟢 STEP 6 — Targeted Alert Dispatch

Only top 3 NGOs get notification:

- Meals available
- Distance
- Expiry time
- Accept button

This avoids:
❌ Spam broadcasting
❌ Alert fatigue

This is ML-driven alert optimization.

🟢 STEP 7 — First Acceptance Locks Assignment

When NGO clicks ACCEPT:

- Request → Assigned
- Other NGOs auto notified as closed

Race condition handled server-side.

🟢 STEP 8 — Live Pickup Status

NGO updates status:

- Accepted

- On the way
- Picked up
- Completed

Provider sees progress in dashboard.

🟢 STEP 9 — Logging for Continuous Learning

After completion, system logs:

- Accepted / rejected
- Response time
- Actual pickup time
- Distance
- Time band
- Cause tag
- Expiry accuracy

Stored in training table.

This becomes dataset for retraining.

🟢 STEP 10 — Fair Provider Scoring

Provider score based on:

- Reporting timeliness
- Expiry accuracy
- Handover readiness
- Data completeness

NOT based on:
❌ Surplus quantity

This avoids penalizing providers unfairly.

🟢 STEP 11 — Model Retraining Cycle

Admin retrains:

- Train on updated logs
- Recalculate accept_rate
- Recalculate reliability
- Save new model.pkl
- Deploy

System improves over time.

This gives:
Adaptive ML system.

🧠 Why This Is Technically Strong

You have:

✔ Event-driven architecture
✔ Real-time ML inference
✔ Classification + Regression
✔ Feature engineering
✔ No data leakage
✔ Fairness constraint
✔ Continuous learning loop
✔ Smart alert optimization

This is not a CRUD app with AI tag.

This is actual ML deployment logic.

🏆 Final System Summary (Easy Version)

When extra food appears:

1. Provider submits request
2. Nearby NGOs filtered
3. ML predicts:
    - Who will accept
    - Who will arrive fastest
4. Ranking engine selects best NGOs
5. Alerts sent
6. First accept gets assignment
7. Pickup tracked
8. Data logged
9. Model retrained
10. System improves

.

🍱 AHARAM SETU

Smart ML-Powered Excess Food Rescue System

🎯 OBJECTIVE

→ Reduce edible food wastage through intelligent routing
→ Connect surplus food providers with the most suitable nearby NGO
→ Minimize coordination delay and food expiry
→ Enable fair, data-driven rescue decisions

🏗️ METHOD / ARCHITECTURE

→ Provider submits surplus food details (location, expiry, cause)
→ System filters nearby NGOs using GPS distance
→ Feature vector built using distance, reliability, response history
→ RandomForest model predicts NGO acceptance probability
→ Smart ranking engine scores NGOs using probability + speed + reliability
→ Top NGOs notified first to avoid alert fatigue
→ First NGO to accept is automatically assigned
→ All outcomes logged for continuous model retraining

✅1️⃣ Acceptance Rate

Definition:
→ Percentage of rescue requests an NGO accepts out of total requests received.

Formula:
Acceptance Rate =
(Number of Accepted Requests) / (Total Requests Received)

✅2️⃣ Reliability Score

Definition:
→ A composite metric measuring how dependable an NGO is.

It includes:
→ Acceptance rate
→ Average response time
→ Past successful pickups
→ Activity consistency

Example (your system idea):

Reliability =
Acceptance Rate × (1 / Avg Response Time) × Past Pickups

📌 What it tells you:
→ How trustworthy and consistent the NGO is overall

Core ML model:

1️⃣ Acceptance Probability Model (Primary Model)

Type: Classification Model
Example Algorithm: RandomForestClassifier

Purpose:
→ Predict probability that an NGO will accept the rescue request quickly.

Input Features:

- distance_km
- ngo_accept_rate
- avg_response_time
- past_pickups
- time_of_day
- day_of_week
- recent_activity_flag

Output
→ Probability score (0–1)

Example:
NGO A → 0.82
NGO B → 0.45

🛠️ TECHNOLOGY / TOOL STACK

→ Frontend built using React / Streamlit
→ Backend developed with FastAPI (Python)
→ ML model implemented using scikit-learn
→ Data stored in PostgreSQL / Firebase
→ Geo-calculations using Haversine / Maps API

💡 IDEA / SOLUTION

→ Replace random calling with intelligent ML-based routing
→ Predict which NGO will accept quickly
→ Rank NGOs based on speed and reliability

→ Optimize rescue decisions in real time
→ Improve system performance through continuous learning

📈 EXPECTED OUTCOME

→ Faster food pickup and reduced expiry
→ Increased NGO acceptance rate
→ Reduced coordination overhead
→ Transparent performance tracking
→ Scalable across cities

🧠 CONCLUSION

→ Food waste is a coordination problem, not a shortage problem
→ Aharam Setu enables real-time intelligent rescue routing
→ Machine Learning ensures smarter decisions over time
→ Surplus food reaches people before it becomes waste